



SEGi
University &
Colleges

University of
Hertfordshire **UH**

SECURE CHAT APPLICATION

TAN JIAJIAN
SCSJ1901781

SV: Ms. Nur Diana Madinah

BACKGROUND

- Online communication is widely used for messaging, file sharing and real-time interaction.
- Cyberattacks, data breaches and unauthorized access have increased the need for secure communication platforms.
- End-to-end encryption (E2EE) helps protect message confidentiality by ensuring that only the sender and receiver can read the messages.
- This project develops ChipChat using RSA-OAEP and AES-GCM to provide secure and private communication.

PROBLEM STATEMENT

1. Lack of Security in Communication Platforms

- Many messaging applications do not provide sufficient protection against cyber threats and unauthorized access.

2. Privacy Concerns and Data Breaches

- Sensitive information such as personal messages and user data may be exposed through data breaches or attacks.

3. Need for Reliable Encryption and Key Management

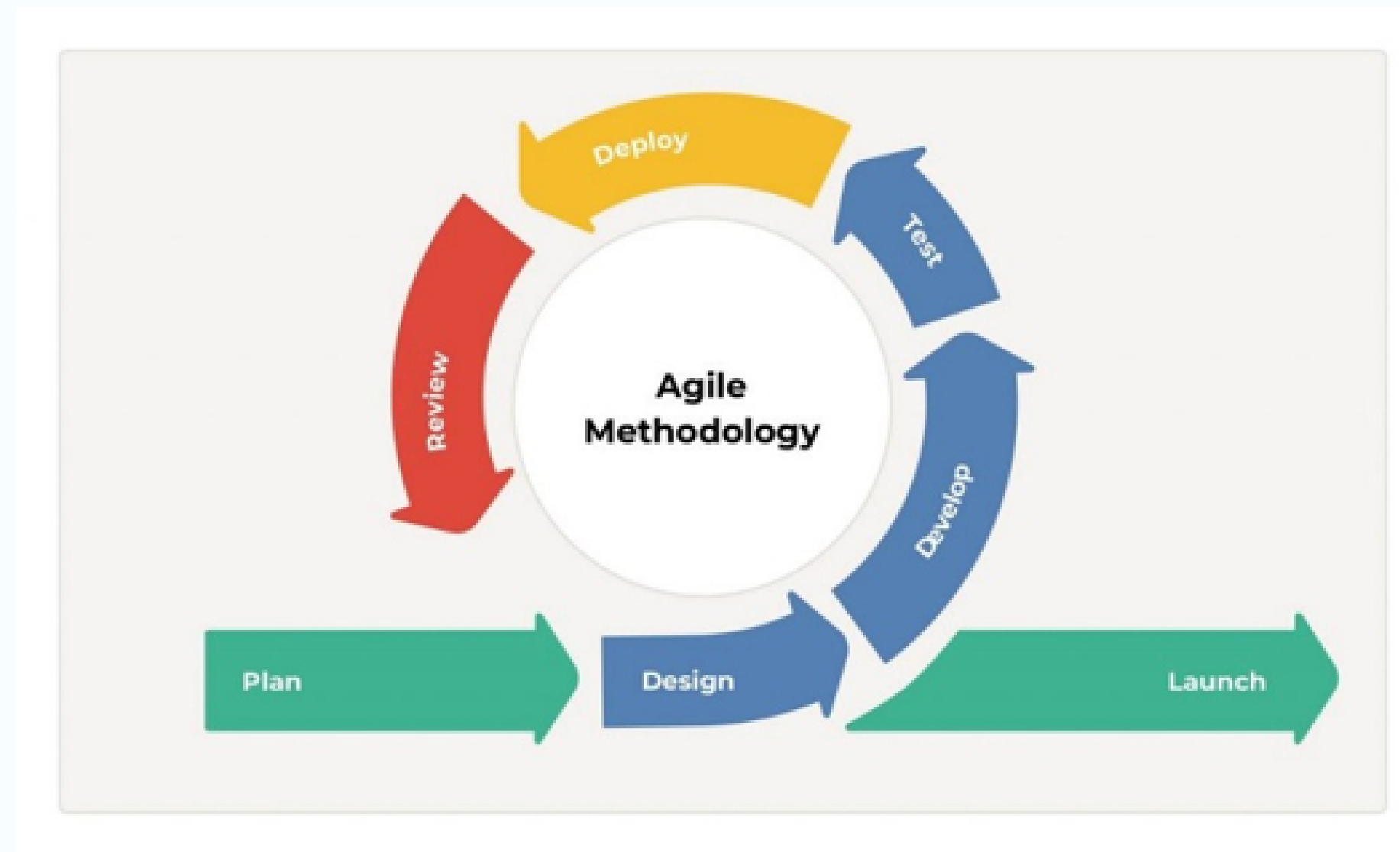
- Secure encryption and proper key management are required to ensure message confidentiality and communication security.



OBJECTIVES

1. To design and develop a secure chat application using end-to-end encryption with RSA-OAEP and AES-GCM cryptographic algorithms.
 2. To ensure message confidentiality, integrity and authenticity through secure key exchange and encryption technologies.
 3. To evaluate the application's performance and user experience in terms of encryption speed, message delivery and ease of use.
- 

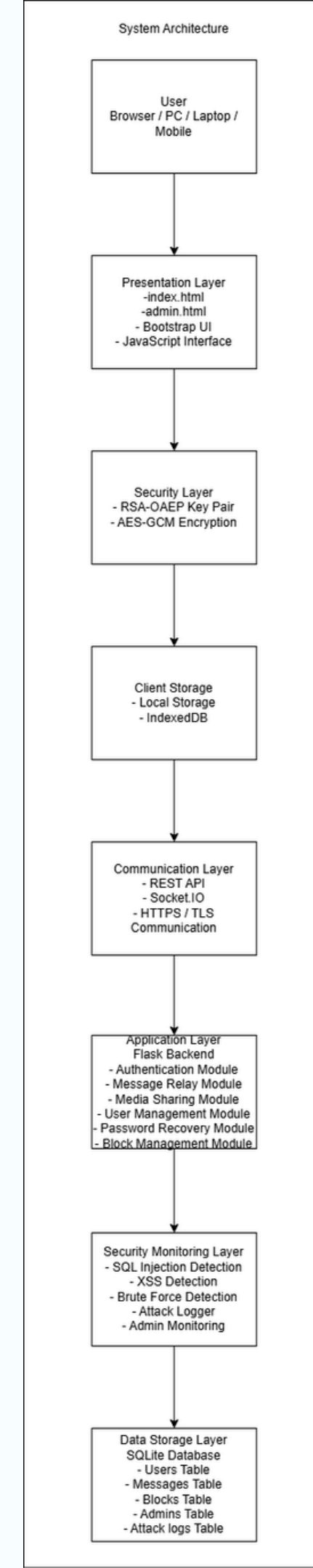
METHODOLOGY



This project uses Agile methodology because it allows development and testing to be performed continuously. Each feature was developed, tested and improved in multiple iterations until the final system was completed.

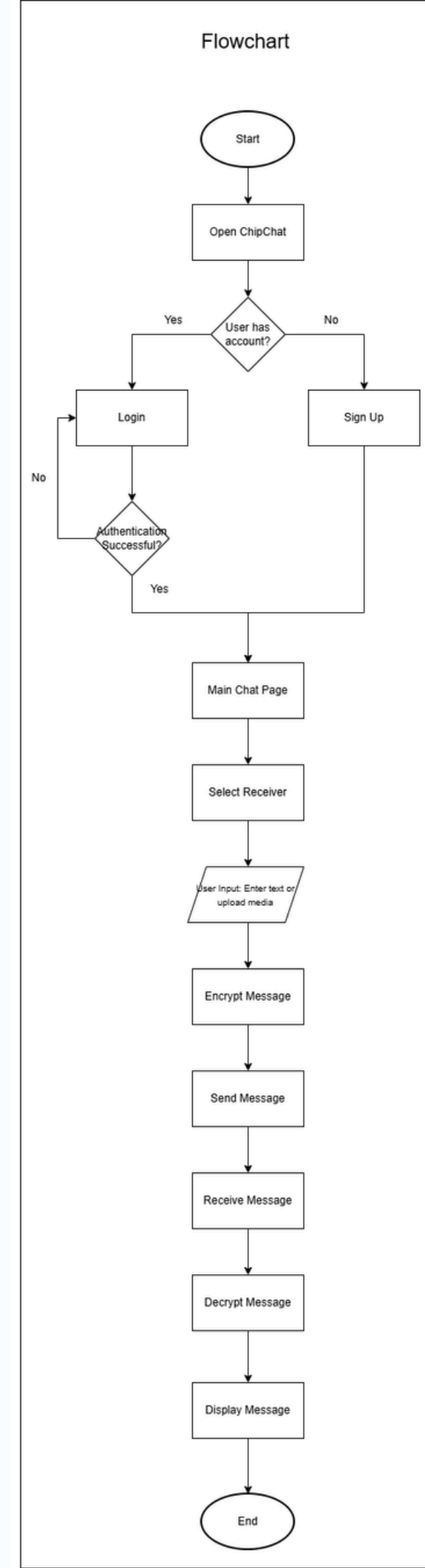
SYSTEM ARCHITECTURE

The ChipChat system follows a layered architecture consisting of the presentation layer, security layer, client storage, communication layer, application layer, security monitoring layer and database layer. This structure separates system functions and improves security, maintainability and scalability.



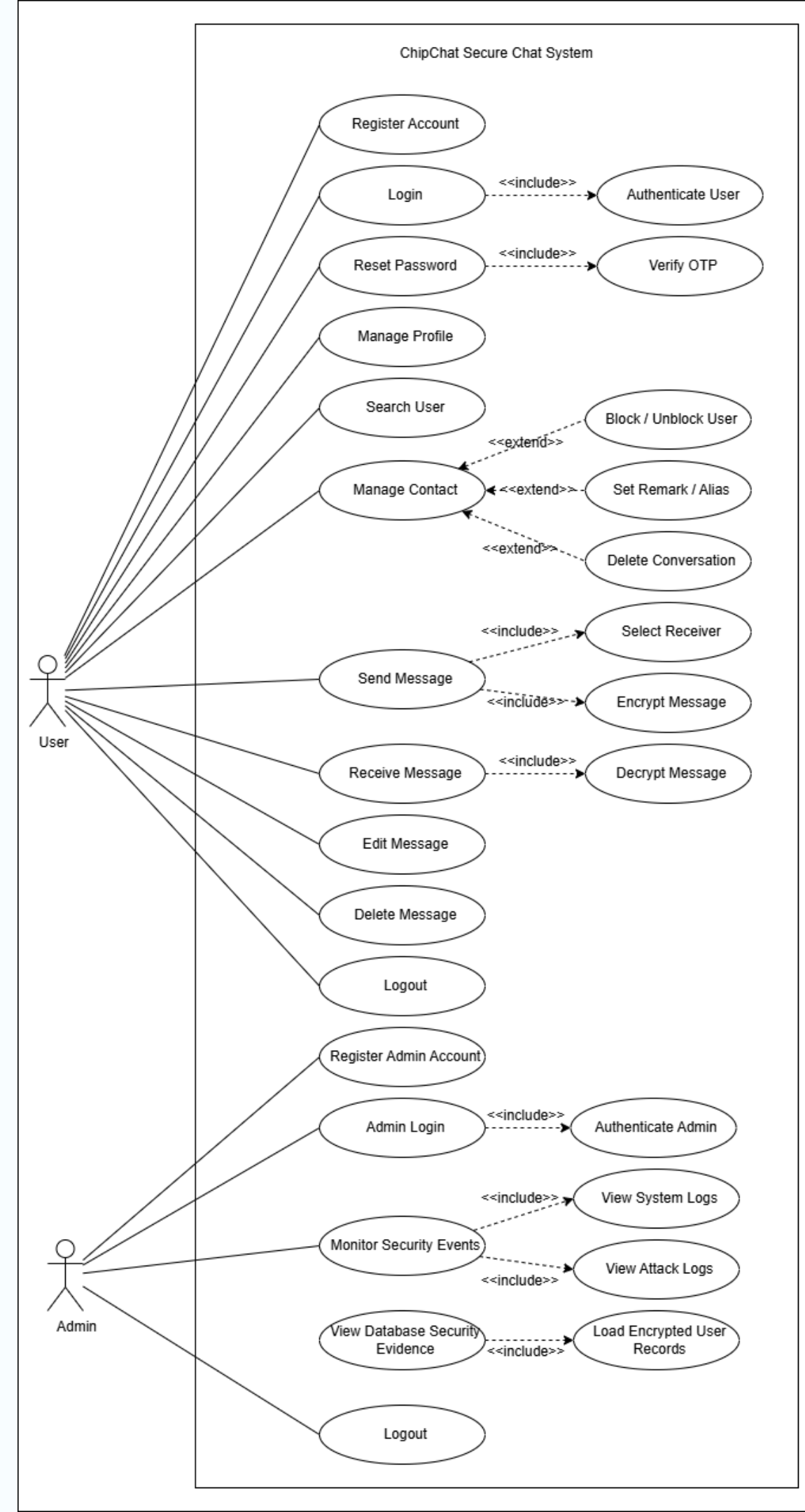
SYSTEM FLOWCHART

The system flow begins with user authentication, followed by receiver selection, message encryption, secure message transmission, message decryption and message display at the recipient side.



USE CASE DIAGRAM

The use case diagram illustrates the interactions between users, administrators and the ChipChat system, including messaging, account management and security monitoring functions.



PERFORMANCE TESTING

The image displays two side-by-side screenshots of the Chrome DevTools Console for the application 'chipchat.today'. Both screenshots show the 'Console' tab with 5 issues listed.

Left Screenshot (Encryption Time):

Issue	Message	Location
2	Encryption Time: 0.30 ms	app.js:356
	Encryption Time: 0.20 ms	app.js:356
	Encryption Time: 0.10 ms	app.js:356
	Encryption Time: 0.30 ms	app.js:356
	Encryption Time: 0.20 ms	app.js:356
2	Encryption Time: 0.30 ms	app.js:356
	Encryption Time: 0.20 ms	app.js:356
	Encryption Time: 0.30 ms	app.js:356

Right Screenshot (Message Delivery Time):

Issue	Message	Location
	Message Delivery Time: 38 ms	app.js:447
2	Message Delivery Time: 37 ms	app.js:447
	Message Delivery Time: 47 ms	app.js:447
	Message Delivery Time: 50 ms	app.js:447
	Message Delivery Time: 36 ms	app.js:447
	Message Delivery Time: 37 ms	app.js:447
	Message Delivery Time: 41 ms	app.js:447
	Message Delivery Time: 70 ms	app.js:447
	Message Delivery Time: 43 ms	app.js:447

Test	Result
Encryption Time	0.10 ms – 0.30 ms
Average Encryption Time	0.25 ms
Delivery Time	36 ms – 70 ms
Average Delivery Time	43.6 ms

The performance testing results show that ChipChat can encrypt messages efficiently and deliver messages with low latency, providing a smooth user experience.

SECURITY TESTING

[10:39:21 PM] **ATTACK** SQL Injection Attempt

TYPE : SQL Injection Attempt
SOURCE : 127.0.0.1
TARGET : /login
PAYLOAD : phone=0183214680, password= ' OR 1=1
ACTION : Request Rejected

[10:40:19 PM] **ATTACK** Cross-Site Scripting (XSS) Attempt

TYPE : Cross-Site Scripting (XSS) Attempt
SOURCE : 127.0.0.1
TARGET : /login
PAYLOAD : phone=0183214680, password=
ACTION : Request Rejected

[5:10:26 PM] **ATTACK** Replay Attack Detected

TYPE : Replay Attack Detected
SOURCE : 0183214680
TARGET : chat_message
PAYLOAD : Duplicate msgId: 1780736486967424242
ACTION : Duplicate Message Rejected

[10:40:50 PM] **ATTACK** Brute Force Login Attempt

TYPE : Brute Force Login Attempt
SOURCE : 127.0.0.1
TARGET : /login
PAYLOAD : Phone: 0183214680 | Failed Attempts: 3
ACTION : Suspicious repeated login failures logged

[10:40:58 PM] **ATTACK** Brute Force Login Attempt

TYPE : Brute Force Login Attempt
SOURCE : 127.0.0.1
TARGET : /login
PAYLOAD : Phone: 0183214680 | Failed Attempts: 4
ACTION : Suspicious repeated login failures logged

Attack Type	Result
SQL Injection	Detected and Blocked
Cross-Site Scripting (XSS)	Detected and Blocked
Brute Force Login Attempt	Detected and Logged
Replay Attack	Detected and Rejected

Security testing showed that ChipChat was able to detect and prevent common cyberattacks including SQL injection, cross-site scripting (XSS), brute-force login attempts and replay attacks. Suspicious activities were automatically blocked or logged for security monitoring purposes.

CONCLUSION

- ChipChat was successfully developed using RSA-OAEP and AES-GCM end-to-end encryption.
- The system achieved secure messaging, media sharing and user authentication functions.
- Testing results showed efficient encryption performance, low message delivery time and successful detection of common cyberattacks.
- The project demonstrates that strong cryptographic techniques can be integrated into a practical communication platform.

The image features a light blue background with four decorative corner elements. Each corner contains a dark blue square with diagonal lines and a medium blue square. The top-left and bottom-right corners have the dark blue square in the foreground, while the top-right and bottom-left corners have the medium blue square in the foreground.

THANK YOU